

Lecture 4: Mathematical Foundations, Sets, and the Asymptotic Complexity of Merge Sort

Yuhong Liu

yuhong.liu@yale.edu

BIS 527: Introduction to Health Data Science

Yale School of Public Health

Lecturer: Harsh Parikh • Fall 2026

TL;DR

Analyzing algorithmic scalability requires an unambiguous mathematical notation. This chapter establishes the set-theoretic apparatus, formal logic quantifiers, and mapping properties necessary to analyze computational procedures. We formalize running time not by elapsed machine clock seconds, but as atomic comparison counts. We introduce asymptotic notations O , Ω , and Θ following standard conventions. Finally, we analyze Merge Sort under the divide-and-conquer paradigm: we derive its exact recurrence relation, unroll it for input cohorts of size 2^k , prove its $O(n \log_2 n)$ upper bound by strong mathematical induction, resolve the cardinality equivalence between natural and rational numbers, and establish the theoretical optimality of comparison sorting.

AI use declaration. We used Gemini Flash to research as well as polish the writing in the notes.

1 Motivation

Modern clinical research and epidemiological registries routinely process cohort registries containing millions of individual biomarker records. A computational procedure designed to sort or partition patient data may execute in fractions of a second on a hospital workstation with sixteen cores, while taking noticeably longer on an embedded diagnostic device. Physical wall-clock runtime varies with hardware configuration, memory caching, programming languages, and background processes.

To evaluate whether a procedure scales as patient registries expand, we count discrete, atomic operations rather than elapsed seconds. Analyzing these operations requires a rigorous mathematical

vocabulary: formal set definitions, logic quantifiers, mapping criteria, and asymptotic growth notations. This chapter builds that foundation and evaluates the divide-and-conquer algorithm Merge Sort.

2 Setup and Notation

Prerequisites. This chapter assumes summation notation and proof by mathematical induction, and assumes no prior familiarity with advanced set theory.

Standard Number Systems. We denote standard numerical universes using blackboard bold type:

- **Natural numbers ($\mathbb{N}$):** $\mathbb{N} := \{0, 1, 2, 3, \dots\}$. By convention in computer science and discrete algorithms, 0 is included as a natural number.
- **Integers ($\mathbb{Z}$):** $\mathbb{Z} := \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$. Negative integers historically arose from bookkeeping and commercial ledgers to denote debits.
- **Rational numbers ($\mathbb{Q}$):** The set of all quotients formed by an integer numerator and a strictly positive integer denominator:

$$\mathbb{Q} := \left\{ x : x = \frac{p}{q}, p \in \mathbb{Z}, q \in \mathbb{N} \setminus \{0\} \right\}. \quad (1)$$

- **Real numbers ($\mathbb{R}$):** The continuous set of all numbers along the real line, encompassing both rational fractions and irrational limits.

These systems form a nested hierarchy of containment:

$$\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}. \quad (2)$$

Set Operations and Notation. A set is a well-defined collection of distinct objects. It may be specified by explicit enumeration, such as $A = \{1, 2, 3\}$, or by set-builder notation $\{x : P(x)\}$, where x represents the candidate element and $P(x)$ denotes the inclusion criterion.

- **Subsets and Supersets:** We write $A \subseteq B$ to assert that every element of A is an element of B . If $A \subseteq B$ and $A \neq B$, then A is a strict subset of B , denoted $A \subset B$, or equivalently $B \supset A$.
- **Set Difference:** The relative complement of B in A , denoted $A \setminus B$, removes from A all elements that belong to B :

$$A \setminus B := \{x : x \in A \text{ and } x \notin B\}. \quad (3)$$

For example, if $A = \{1, 2, 3\}$ and $B = \{3, 4\}$, then $A \setminus B = \{1, 2\}$.

- **Cardinality:** We write $|A|$ for the cardinality, or total count of elements, in set A .

Array Slicing Conventions. Following the indexing conventions in [Cormen et al. \(2022\)](#), lists are indexed from zero, and slices exclude their right endpoint:

- $A[i:j]$ denotes the contiguous slice $\{A[i], A[i+1], \dots, A[j-1]\}$, which is empty when $i \geq j$.
- $A[:j]$ denotes the prefix $\{A[0], A[1], \dots, A[j-1]\}$.
- $A[i:]$ denotes the suffix $\{A[i], A[i+1], \dots, A[n-1]\}$ for a list of n elements.

3 Set Relations, Logic Quantifiers, and Functions

Membership and Logic Quantifiers. Membership is stated using $\in$. To state that the value 2 belongs to the natural numbers, we write $2 \in \mathbb{N}$, which evaluates to true. Mathematical statements rely on two primary quantifiers:

- **Existential Quantifier ($\exists$):** Read as “there exists”. The claim

$$\exists x \text{ s.t. } x \in \mathbb{N} \text{ and } x \text{ is divisible by } 2 \quad (4)$$

is true because $x = 2$ satisfies the condition.

- **Universal Quantifier ($\forall$):** Read as “for all” or “for every”. Quantifier order affects the meaning of a claim. Let S denote the set of students in a lecture cohort. The claim

$$\forall s \in S, \exists y \in \mathbb{R} \text{ s.t. } y \text{ is the favorite number of } s \quad (5)$$

asserts that every individual student holds at least one favorite number. Reversing the quantifiers produces:

$$\forall y \in \mathbb{N}, \exists s \in S \text{ s.t. } y \text{ is the favorite number of } s. \quad (6)$$

This second claim is false: $|\mathbb{N}| = \infty$ while the cohort size $|S|$ is finite, so no finite set of students can cover every natural number.

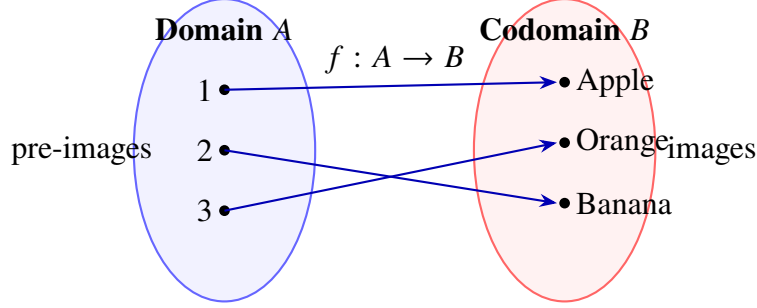


Figure 1: A function $f : A \rightarrow B$ mapping every pre-image in domain A to an image in codomain B .

Mappings and Functions. Consider two sets A and B . A mapping creates an association between elements of A and elements of B .

Definition 1 (Function). A *function* $f : A \rightarrow B$ from domain A to codomain B is a relation that assigns to each element $x \in A$ (the pre-image) exactly one element $f(x) \in B$ (the image).

If any pre-image maps to multiple images, the relation is not a function. In statistical modelling, affine functions $f(x) = ax + b$ map inputs from pre-image x using parameters $a, b \in \mathbb{R}$. In algorithmic complexity, we analyze functions mapping problem size to computational operations: $T : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$.

4 Computational Complexity and Asymptotic Bounds

Operation Counts Over Wall-Clock Time. To analyze an algorithm $\mathcal{A}$, we define runtime $T(n)$ as the count of fundamental atomic operations executed on an input of size n . Atomic operations include:

- Comparisons between two scalar keys (such as $A[i] > A[j]$).
- Arithmetic operations (such as addition, subtraction, or multiplication).
- Data assignment and memory writes.

As problem size n grows, constant scaling factors and lower-order terms become negligible compared to the dominant growth rate. We formalize this using the asymptotic notations standard in the algorithms literature (Cormen et al., 2022; Knuth, 1998).

Definition 2 (Asymptotic Notations). Let $T(n)$ and $f(n)$ be functions mapping $\mathbb{N}$ to $\mathbb{R}_{\geq 0}$.

1. **Big- O (Asymptotic Upper Bound):** As defined by Cormen et al. (2022), $T(n) \in O(f(n))$ if there exist positive constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$0 \leq T(n) \leq c \cdot f(n) \quad \text{for all } n \geq n_0. \quad (7)$$

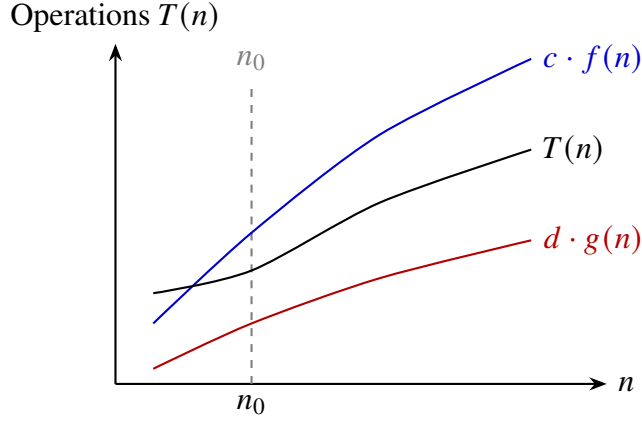


Figure 2: Geometric characterization of asymptotic bounds: for all input dimensions $n \geq n_0$, the actual operation cost $T(n)$ is bounded above by $c \cdot f(n)$ and below by $d \cdot g(n)$ (Cormen et al., 2022).

2. **Big- Ω (Asymptotic Lower Bound):** $T(n) \in \Omega(g(n))$ if there exist positive constants $d > 0$ and $n_0 \in \mathbb{N}$ such that

$$0 \leq d \cdot g(n) \leq T(n) \quad \text{for all } n \geq n_0. \quad (8)$$

3. **Big- Θ (Asymptotically Tight Bound):** $T(n) \in \Theta(f(n))$ if and only if $T(n) \in O(f(n))$ and $T(n) \in \Omega(f(n))$ simultaneously (Cormen et al., 2022). That is, there exist positive constants $c_1, c_2 > 0$ and $n_0 \in \mathbb{N}$ such that

$$c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n) \quad \text{for all } n \geq n_0. \quad (9)$$

5 Merge Sort: Recurrence and Complexity Analysis

Merge Sort applies the divide-and-conquer paradigm (Cormen et al., 2022):

1. **Divide:** Partition the list of n records into two contiguous halves of lengths $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$.
2. **Conquer:** Recursively sort both sub-lists.
3. **Combine:** Merge the two sorted halves into a single ordered sequence.

The Comparison Cost of Merging. Let the two sorted sub-lists have lengths a and b . To merge them, the procedure compares the smallest unplaced element of each sub-list and moves the smaller into the output buffer.

Each comparison places at least one element. The process halts as soon as one sub-list is exhausted; the remaining elements of the other sub-list are then copied directly without further

Algorithm 1 Merge Sort

```
1: function MERGESORT( $A, n$ )
2:   if  $n \leq 1$  then
3:     return  $A$                                  $\triangleright$  Base case: lists of length 0 or 1 are already sorted
4:   end if
5:    $mid \leftarrow \lfloor n/2 \rfloor$ 
6:    $L \leftarrow \text{MERGESORT}(A[0:mid], mid)$ 
7:    $R \leftarrow \text{MERGESORT}(A[mid:n], n - mid)$ 
8:   return MERGE( $L, R$ )
9: end function
```

comparisons. In the worst case, one sub-list empties on the final step, requiring at most $a + b - 1$ comparisons.

For an array of length n split into two sub-lists of size $n/2$, the combine step costs:

$$\text{Merge Cost} \leq \frac{n}{2} + \frac{n}{2} - 1 = n - 1 \text{ comparisons.} \quad (10)$$

Recurrence Formulation. Let $T(n)$ denote the maximum number of comparisons performed by Merge Sort on an input of size n . The algorithm yields the recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + (n - 1), \quad \text{with base case } T(1) = 0. \quad (11)$$

5.1 Recurrence Unrolling for Powers of Two

Suppose $n = 2^k$ for an integer $k \in \mathbb{N}$, so that $k = \log_2 n$. Unrolling Equation (11) step by step:

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n - 1 \\ &= 2\left[2T\left(\frac{n}{4}\right) + \frac{n}{2} - 1\right] + n - 1 \\ &= 4T\left(\frac{n}{4}\right) + (n - 2) + (n - 1) \\ &= 4\left[2T\left(\frac{n}{8}\right) + \frac{n}{4} - 1\right] + (n - 2) + (n - 1) \\ &= 8T\left(\frac{n}{8}\right) + (n - 4) + (n - 2) + (n - 1). \end{aligned}$$

Continuing this expansion across k levels of recursion gives:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + \sum_{m=0}^{k-1} (n - 2^m). \quad (12)$$

At the terminal level, $n/2^k = 1$, and evaluating the base case yields $T(1) = 0$. Summing the remaining terms gives:

$$\begin{aligned}
T(n) &= \sum_{m=0}^{k-1} n - \sum_{m=0}^{k-1} 2^m \\
&= k \cdot n - (2^k - 1) \\
&= n \log_2 n - n + 1.
\end{aligned} \tag{13}$$

Because $-n + 1 \leq 0$ for all $n \geq 1$:

$$T(n) \leq n \log_2 n. \tag{14}$$

This confirms that $T(n) \in O(n \log_2 n)$.

6 Formal Induction Proof of Complexity

We formalize the upper bound across all cohort sizes by mathematical induction, following the analysis in [Knuth \(1998\)](#).

Theorem 1 (Comparison Cost of Merge Sort). *Let $T(n)$ satisfy $T(n) = 2T(n/2) + n - 1$ for all $n = 2^k$ with $k \geq 1$, and $T(1) = 0$. Then for all $n = 2^k$:*

$$T(n) \leq n \log_2 n. \tag{15}$$

Proof. We proceed by strong induction on $k \geq 0$, where $n = 2^k$.

Base case ($k = 0$). When $k = 0$, $n = 2^0 = 1$. The base case of the procedure performs no comparisons:

$$T(1) = 0 \leq 1 \cdot \log_2 1 = 0. \tag{16}$$

The base claim holds.

Inductive hypothesis. Assume the claim holds for all integers $j < k$. Specifically, assume that for $m = 2^{k-1} = n/2$, the number of comparisons satisfies $T(n/2) \leq (n/2) \log_2(n/2)$.

Inductive step. For $n = 2^k$, expanding $T(n)$ via Equation (11) gives:

$$\begin{aligned}
T(n) &= 2T\left(\frac{n}{2}\right) + n - 1 \\
&\leq 2\left[\frac{n}{2}\log_2\left(\frac{n}{2}\right)\right] + n - 1 \\
&= n(\log_2 n - \log_2 2) + n - 1 \\
&= n(\log_2 n - 1) + n - 1 \\
&= n\log_2 n - n + n - 1 \\
&= n\log_2 n - 1 \\
&< n\log_2 n.
\end{aligned}$$

The induction step holds. By strong induction, $T(n) \leq n\log_2 n$ for all $n = 2^k$. $\square$

For general cohort sizes n that are not exact powers of two, the problem can be bounded by embedding n into the next power of two, $n \leq 2^{\lceil \log_2 n \rceil} < 2n$. Standard divide-and-conquer analysis then guarantees $T(n) \in O(n\log_2 n)$ (Cormen et al., 2022).

7 Worked Example

Consider sorting $n = 8$ patient systolic blood pressure measurements:

$$A = \langle 140, 118, 162, 124, 155, 130, 170, 110 \rangle. \quad (17)$$

Here $n = 2^3$, so $k = 3$. Table 1 traces each merge level.

Level (m)	Merges Performed	Subarray Size	Maximum Comparisons
1	4 merges of size 1 into size 2	2	$4 \times (2 - 1) = 4$
2	2 merges of size 2 into size 4	4	$2 \times (4 - 1) = 6$
3	1 merge of size 4 into size 8	8	$1 \times (8 - 1) = 7$

Table 1: Maximum comparisons per recursion level for Merge Sort on an array of size $n = 8$.

Summing across all levels yields the total comparisons:

$$T(8) \leq 4 + 6 + 7 = 17. \quad (18)$$

Applying the exact closed formula from Equation (13):

$$T(8) = 8\log_2 8 - 8 + 1 = 8(3) - 8 + 1 = 24 - 8 + 1 = 17. \quad (19)$$

The analytical formula matches the exact trace, and both satisfy $T(8) \leq 8 \log_2 8 = 24$.

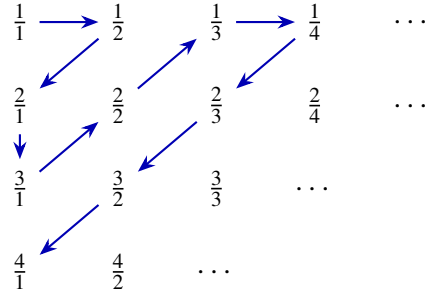
8 Beyond the Lecture

Investigating $|\mathbb{N}| = |\mathbb{Q}|$. During lecture, the claim that the size of the natural numbers equals the size of the rational numbers was left as an open question. While $\mathbb{N} \subset \mathbb{Q}$ and the rationals are dense on the real line, the mathematical claim

$$|\mathbb{N}| = |\mathbb{Q}| \quad (20)$$

is rigorously **True**.

Two sets have identical cardinality if and only if there exists a bijection (a one-to-one and onto mapping) between them. As established by [Cantor \(1874\)](#), any countable union of countable sets is countable. We establish this equivalence by organizing the positive rationals $\mathbb{Q}^+$ into a two-dimensional grid where entry (p, q) represents p/q :



Traversing diagonally along antidiagonals (where $p + q = k$) yields an exhaustive listing:

$$\frac{1}{1}, \frac{1}{2}, \frac{2}{1}, \frac{3}{1}, \frac{2}{2}, \frac{1}{3}, \frac{1}{4}, \frac{2}{3}, \dots \quad (21)$$

Skipping fractions that are not in lowest terms (such as $2/2 = 1/1$) pairs each distinct positive rational with a unique natural number index. Interleaving zero and negative rationals extends this into a bijection between $\mathbb{N}$ and $\mathbb{Q}$ ([Cantor, 1874](#)). Both sets share the same transfinite cardinality: $\aleph_0$ (aleph-null).

Optimality of Merge Sort. Algorithms such as Bubble Sort remove at most one out-of-order element per pass and incur $\Theta(n^2)$ comparisons in the worst case ([Cormen et al., 2022](#)). In contrast, Merge Sort recursively halves the cohort and recombines the sub-lists in linear time, yielding $T(n) \in \Theta(n \log_2 n)$ comparisons ([Knuth, 1998](#)).

Any comparison sort can be modelled as a binary decision tree with $n!$ leaves corresponding to possible permutations. Because a binary tree of height h has at most 2^h leaves, distinguishing

among all $n!$ orderings requires:

$$h \geq \log_2(n!) = \sum_{i=1}^n \log_2 i \geq \frac{n}{2} \log_2 \left(\frac{n}{2}\right) = \Omega(n \log_2 n). \quad (22)$$

As proved by [Cormen et al. \(2022\)](#), every comparison-based sorting procedure requires $\Omega(n \log_2 n)$ comparisons in the worst case. Merge Sort achieves this bound, making it asymptotically optimal.

9 Summary

- Standard number systems form a strict containment hierarchy $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$, with $0 \in \mathbb{N}$ by computer science convention.
- Set differences $A \setminus B$ remove from A all elements in B . A function $f : A \rightarrow B$ maps every element in domain A to a single element in codomain B .
- Quantifier ordering alters meaning: universal coverage claims over infinite sets cannot be satisfied by finite cohorts.
- As proven by [Cantor \(1874\)](#), the set of natural numbers and the set of rational numbers have equal cardinality: $|\mathbb{N}| = |\mathbb{Q}| = \aleph_0$.
- Algorithmic work is evaluated by counting atomic operations rather than elapsed seconds, decoupling complexity from specific hardware architectures.
- Asymptotic bounds formalized by [Cormen et al. \(2022\)](#) define performance rates: O provides an upper bound, Ω a lower bound, and Θ a tight bound.
- Merging two sorted lists of lengths a and b requires at most $a + b - 1$ comparisons.
- The divide-and-conquer recurrence $T(n) = 2T(n/2) + n - 1$ unrolls to $n \log_2 n - n + 1$, establishing that Merge Sort has comparison cost $O(n \log_2 n)$, which is asymptotically optimal among comparison-based sorts ([Cormen et al., 2022](#); [Knuth, 1998](#)).

Further Reading

[Cormen et al. \(2022\)](#) provide standard treatments of asymptotic notations in Chapter 3, divide-and-conquer recurrences and the Master Theorem in Chapter 4, and the $\Omega(n \log_2 n)$ lower bound for comparison sorting in Chapter 8. [Knuth \(1998\)](#) offers detailed analysis of merge algorithms, exact comparison bounds, and stable sorting variants. For the foundations of transfinite cardinalities and diagonal enumeration, see [Cantor \(1874\)](#).

References

- Cantor, G. (1874). Über eine eigenschaft des inbegriffes aller reellen algebraischen zahlen. *Journal für die reine und angewandte Mathematik*, 1874(77):258–262.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4th edition.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2nd edition.